LoRA + PEFT

Parameter-Efficient Fine-Tuning

Fine-Tune a 70B Model on a Single GPU

WEEK 2 · DAY 9 **Tuesday, July 14 2026** **Day 9 of 84**

#	Topic	What You Learn
01	The Fine-Tuning Problem	Why full FT costs $3,300+ per experiment
02	Low-Rank Hypothesis	Why ΔW lives in a tiny subspace
03	LoRA Mathematics	W' = W + (α/r)×B×A — the full derivation
04	LoRA Hyperparameters	Rank r, alpha, target modules, dropout
05	PEFT Library	get_peft_model(), print_trainable_parameters()
06	QLoRA	NF4 base + LoRA adapters: 70B on 1× A100
07	Other PEFT Methods	Prefix Tuning, IA3, Prompt Tuning comparison
08	LoRA Merge	merge_and_unload() → production pipeline
09	FAANG in Production	Meta · Google AdaLoRA · Microsoft · Amazon · Netflix
10	Text-to-SQL Thread	Piece 2: SQL domain adaptation with LoRA
11	Architect's Lens	LoRA vs RAG vs prompting decision framework
12	Hands-On Exercises	Parameter counting · dataset builder · merge

TEXT-TO-SQL CAPSTONE THREAD — LoRA is the fine-tuning layer of the Text-to-SQL pipeline. Today you learn HOW to adapt a base model to your SQL domain without full retraining. The pipeline so far: Day 6 (SQLOutput schema) → Day 8 (quantized inference) → Day 9 (domain adaptation via LoRA).

Section 1 — The Full Fine-Tuning Memory Problem

You have LLaMA 3 70B. You want to fine-tune it for SQL generation. Here is the complete memory bill for standard full fine-tuning:

Component	Size	Why
Model weights (FP16)	140 GB	70B params × 2 bytes
Adam optimizer — mean m	280 GB	One FP32 copy per parameter
Adam optimizer — variance v	280 GB	One FP32 copy per parameter
Gradients	140 GB	FP32, same shape as weights
Total	~840 GB	11× A100 80GB GPUs minimum

> *At $30/hour for 11× A100s, one 10-hour training run costs $3,300. 20 hyperparameter experiments = $66,000. LoRA reduces the trainable parameter count to 0.5%, cutting the optimizer and gradient memory to ~300 MB. Total: 35 GB (1× A100 with QLoRA).*

Section 2 — The Low-Rank Hypothesis: Why LoRA Works

Paper: *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning* (Aghajanyan et al., 2021).

When you fine-tune a pre-trained model, you compute weight updates ΔW. The key empirical discovery: Δ**W is low-rank.** The meaningful updates live in a much smaller subspace — typically 4 to 64 directions out of thousands. The model already knows almost everything; fine-tuning shifts just a few directions in weight space.

Intuition

The pre-trained model knows 99% of what it needs for SQL generation. It already knows SQL syntax, schema concepts, JOIN logic. What it lacks is the specific mapping from natural language questions in your domain to correct SQL for your schema. That specific mapping lives in a low-dimensional subspace of the full weight space.

Aghajanyan et al. (2021) showed that fine-tuning RoBERTa-large on MRPC requires only 200 effective dimensions out of 356M parameters — 0.00006% of the full parameter space. The fine-tuning signal is extremely concentrated.

Section 3 — LoRA Mathematics

Paper: *LoRA: Low-Rank Adaptation of Large Language Models* (Hu et al., 2021, Microsoft Research)

The Core Decomposition

```
W' = W + Delta_W = W + (alpha / r) x B x A

Where:
  W in R^(d x k)  -- original frozen weight matrix (not trained)
  B in R^(d x r)  -- trainable, initialized to ZERO at start
  A in R^(r x k)  -- trainable, initialized with random Gaussian
  r               -- rank (hyperparameter: typically 4-64)
  alpha           -- scaling factor (typically 2r)

At initialization: Delta_W = B x A = 0 x A = 0
  -> LoRA model starts IDENTICAL to base model
  -> Ensures stable early training
```

Why B=0 at Initialization?

At the start of training, $\Delta W = B \times A = 0 \times A = 0$. The LoRA model produces exactly the same outputs as the unmodified base model. This ensures the adapter starts from a neutral position and learns from there, rather than introducing random noise into the model's outputs.

Parameter Count Comparison

For a 4096×4096 attention matrix with rank r=16:

```
Full fine-tuning:   d x k = 4096 x 4096 = 16,777,216 parameters
LoRA r=16:          d*r + r*k = 4096*16 + 16*4096 = 131,072 parameters
Savings:            98.4% reduction in trainable parameters for this matrix

For all attention layers in LLaMA 3 8B (Q, K, V, O x 32 layers):
Full fine-tuning:   4 x 32 x 16.7M = 2.14B trainable parameters
LoRA r=16 (attn):  4 x 32 x 131K  = 16.8M trainable parameters (0.78%)
```

Forward Pass During Training

```
# Standard frozen forward pass:
h = x @ W.T                     # FP16/BF16 -- no gradient computed

# LoRA adapter (trainable):
lora_out = x @ A.T @ B.T * (alpha / r)   # gradients flow here

# Combined output:
output = h + lora_out
```

```
# During training:  gradients update only B and A
# During inference: output = x @ (W + alpha/r * B @ A).T
#                          = x @ W'.T
```

> *The base model weights W are completely frozen — they receive no gradient updates. Only B (d×r) and A (r×k) receive gradients. The 140 GB base model needs NO optimizer states. Only the 33 MB adapter does.*

Section 4 — LoRA Hyperparameters

Rank (r) — The Most Important Choice

Rank r	Trainable %	Recommended For
4	~0.2%	Classification, extraction, narrow single tasks
8	~0.4%	SQL generation, summarisation — good starting point
16	~0.8%	RECOMMENDED DEFAULT — covers most fine-tuning tasks
32	~1.5%	Style/personality changes, multilingual adaptation
64	~3%	Complex knowledge transfer, approaches full FT quality
128	~6%	Diminishing returns vs r=64 in most benchmarks

Practical rule: Start with r=16. If quality is insufficient after training, double to r=32 before changing anything else.

Alpha (α) — Scaling Factor

Alpha controls the magnitude of the LoRA update. The actual update applied is (α/r)×BA. Setting α = 2r means the scaling factor is 2.0.

```
Common convention:  alpha = 2*r  -> scaling = alpha/r = 2.0
Conservative:       alpha = r    -> scaling = 1.0
Aggressive:         alpha = 4*r  -> scaling = 4.0

# For SQL (narrow task):     alpha=32 with r=16  (scaling=2.0)
# For style change (broad):  alpha=64 with r=16  (scaling=4.0)
```

Target Modules — Which Weights to Adapt

```
# Option 1: Attention only — fewer params, faster training
target_modules = ["q_proj", "k_proj", "v_proj", "o_proj"]
# ~0.5% of params; good for format/style tasks

# Option 2: Attention + Feed-Forward Network (recommended for SQL)
target_modules = [
    "q_proj", "k_proj", "v_proj", "o_proj",    # attention
    "gate_proj", "up_proj", "down_proj",         # FFN
]
# ~1.5% of params; better for knowledge-intensive tasks
# FFN layers store factual knowledge and pattern associations
```

Dropout

- LoRA dropout applied to adapter outputs only (not the base model)
- 0.05–0.10 for datasets >1,000 examples
- 0.0 for very small datasets (<500 examples) — need every gradient signal

Section 5 — PEFT Library: LoRA in Practice

PEFT (Parameter-Efficient Fine-Tuning) is the HuggingFace library implementing LoRA, QLoRA, Prefix Tuning, IA3, and more. It wraps any transformers model with a single function call.

Standard LoRA Setup

```
# pip install peft transformers accelerate trl
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType
import torch

# Load base model in BF16
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# Configure LoRA
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,             # alpha = 2r
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_dropout=0.05,
    bias="none",               # do not train bias terms
    task_type=TaskType.CAUSAL_LM,
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 41,943,040 || all params: 8,072,204,288 || trainable%: 0.5196%
```

Training with SFTTrainer

```
from transformers import TrainingArguments
from trl import SFTTrainer

training_args = TrainingArguments(
    output_dir="./sql-lora-adapter",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,  # effective batch = 16
    learning_rate=2e-4,             # higher LR safe with fewer params
```

```
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    bf16=True,
    logging_steps=50,
    save_steps=500,
    save_total_limit=2,
    report_to="none",
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,    # (instruction, response) pairs
    tokenizer=tokenizer,
    max_seq_length=2048,
    dataset_text_field="text",
)

trainer.train()
trainer.save_model()   # saves ONLY the adapter -- ~33 MB for r=16
```

LoRA learning rate is typically 5-10x higher than full fine-tuning. 2e-4 is standard (vs 2e-5 for full FT). Because only 0.5% of parameters are trained, each update can be more aggressive without destabilizing the frozen base weights.

Section 6 — QLoRA: Day 8 + Day 9 Combined

Paper: *QLoRA: Efficient Finetuning of Quantized LLMs* (Dettmers et al., 2023). QLoRA combines NF4 quantization (Day 8) with LoRA adapters (today) to make 70B fine-tuning possible on a single A100 80GB GPU.

Memory Comparison: Full FT vs LoRA vs QLoRA

Approach	Model	Optimizer	Gradients	Total	GPUs
Full FT FP16	140 GB	280 GB	140 GB	~560 GB	7x A100
LoRA FP16 r=16	140 GB	~0.3 GB	~0.3 GB	~141 GB	2x A100
QLoRA NF4 r=16	35 GB	~0.3 GB	~0.3 GB	~36 GB	1x A100

QLoRA Code

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
import torch

# Step 1: Load base model in NF4 (Day 8 technique)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
)
base_model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-70B-Instruct",
    quantization_config=bnb_config,
    device_map="auto",
)

# Step 2: Enable gradient checkpointing for k-bit training
base_model = prepare_model_for_kbit_training(base_model)
# gradient checkpointing: recompute activations during backward pass
# saves 60-70% activation memory at cost of 30-40% slower training

# Step 3: Attach LoRA adapters
lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05, bias="none", task_type="CAUSAL_LM",
)
```

```
model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable params: 163,577,856 || all params: 70,566,502,400 || trainable%: 0.2318%
```

QLoRA is the most important technique in practical LLM fine-tuning today. It enables domain adaptation of frontier-scale models (70B+) on infrastructure that costs $2-3/hour. Before QLoRA, fine-tuning 70B required 7+ A100s ($25-30/hour). QLoRA reduced the barrier by 10x.

Section 7 — Other PEFT Methods

LoRA dominates production use. Three other methods are worth knowing conceptually — they appear in papers and some specialized deployments.

Prefix Tuning (Li and Liang, 2021)

Prepend trainable 'prefix' tokens to the Key and Value tensors in every attention layer. The prefix tokens are not real words — they are continuous vectors in embedding space that steer attention behavior.

- Parameters: ~0.1% of model
- Use case: generation tasks where you want to shift output style
- Drawback: prefix tokens consume context window space at inference

IA3 — Infused Adapter (Liu et al., 2022)

Instead of adding matrices, IA3 multiplies the key, value, and FFN activations by learned element-wise vectors. Extremely parameter-efficient.

- Parameters: ~0.01% of model
- Use case: few-shot adaptation with minimal compute
- Tradeoff: lower quality ceiling than LoRA at equivalent tasks

Prompt Tuning (Lester et al., 2021, Google)

Add trainable tokens to the input embedding layer only. The simplest PEFT method — works well only at 11B+ scale on encoder-decoder models like T5. Largely superseded by LoRA for decoder-only models (LLaMA, Mistral, GPT).

Comparison Table

Method	Trainable %	Quality	Overhead	Best For
Full FT	100%	Highest	None	Critical quality, budget available
LoRA r=64	~2%	Excellent	None*	Most fine-tuning tasks
LoRA r=16	~0.5%	Very Good	None*	SQL, chat, classification
QLoRA r=16	~0.23%	Very Good	None*	70B on single GPU
Prefix Tuning	~0.1%	Good	Context	Style shift in generation

IA3	~0.01%	Moderate	Minimal	Extreme compute limits
Prompt Tuning	~0.01%	Lower	None	Simple domain shift (T5)

* None after merge_and_unload() — see Section 8.

Section 8 — LoRA Merge: From Adapter to Production

After training, the LoRA adapter (B, A matrices) is separate from the base model. At inference time you have three options:

Option A: Adapter Inference (Keep Separate)

```
from peft import PeftModel

model = AutoModelForCausalLM.from_pretrained("base-model")
model = PeftModel.from_pretrained(model, "sql-lora-adapter")
# Pro: can swap adapters at runtime (multiple tasks, one base model)
# Con: tiny adapter computation overhead per forward pass
```

Option B: Merge and Unload (Production Standard)

```
from peft import PeftModel
import torch

base_model = AutoModelForCausalLM.from_pretrained(
    "base-model", torch_dtype=torch.float16)
peft_model = PeftModel.from_pretrained(base_model, "sql-lora-adapter")

# Merge: W' = W + (alpha/r) * B @ A, baked permanently into W
merged = peft_model.merge_and_unload()
merged.save_pretrained("sql-llama-8b-merged")
tokenizer.save_pretrained("sql-llama-8b-merged")
# Pro: zero inference overhead, same model size as base
# Con: cannot swap adapters; one merged model per task
```

Option C: Merge then Quantize (Recommended Production Pipeline)

```
# The complete Text-to-SQL production pipeline:
#
# 1. QLoRA fine-tune (today, Day 9):
#    base LLaMA 70B NF4 + LoRA r=16 -> sql-lora-adapter/ (163M params, ~330 MB)
#
# 2. Merge (Option B above):
#    W' = W + BA -> sql-llama-70b-merged/ (full FP16, 140 GB)
#
# 3. AWQ quantize (Day 8 technique):
#    from awq import AutoAWQForCausalLM
```

```
#    model = AutoAWQForCausalLM.from_pretrained("sql-llama-70b-merged")
#    model.quantize(tokenizer, quant_config={"w_bit": 4, "q_group_size": 128})
#    model.save_quantized("sql-llama-70b-awq4bit")  # ~38 GB
#
# 4. Serve with vLLM (Day 11):
#    vllm serve sql-llama-70b-awq4bit --quantization awq
#    # 70B SQL-specialized model at <200ms latency on 1x A100 80GB
```

Option C is the architecture that powers production Text-to-SQL systems. QLoRA makes domain adaptation affordable (~$50 for 70B, 10 hours). Merging eliminates adapter overhead. AWQ quantization halves serving cost. The full pipeline: fine-tune for $50, serve at $2/hour on one A100.

Section 9 — FAANG in Production

Meta — LoRA as the Standard for LLaMA Adaptation

Meta publishes LoRA recipes alongside every LLaMA release. Their internal tooling for adapting LLaMA to product-specific domains uses LoRA with r=32-64 for most tasks. The open-source community's entire fine-tuning ecosystem is built on this: thousands of LLaMA variants on HuggingFace use LoRA adapters trained for specific domains (medical, legal, code, SQL, multilingual).

Google — AdaLoRA: Adaptive Rank Allocation

Google Brain published AdaLoRA (Zhang et al., 2023) — an extension that automatically determines optimal rank for each weight matrix using importance scores derived from the singular value decomposition of ΔW. More important matrices (typically early and late attention layers) get higher rank; less important ones get lower rank. AdaLoRA achieves equivalent quality to LoRA with 30% fewer total parameters by investing rank where it matters most.

Microsoft — LoRA's Origin + Azure Fine-Tuning

LoRA was published by Microsoft Research (Hu et al., 2021). Microsoft Azure OpenAI's fine-tuning endpoint for GPT-4o uses LoRA under the hood — when you call POST /fine-tuning/jobs, you submit a LoRA training job. The adapter is kept server-side and applied at inference time per customer namespace. Microsoft also ships LoRA fine-tuning in Azure Machine Learning as a managed capability with automatic instance provisioning.

Amazon — SageMaker JumpStart LoRA

Amazon SageMaker JumpStart includes LoRA fine-tuning as a first-class feature. The JumpStartEstimator with hyperparameters={'peft_type': 'lora'} launches a managed QLoRA training job with automatic instance selection based on model size. Amazon Bedrock's custom model import also accepts LoRA-fine-tuned models and handles the merge step internally before serving.

Netflix — Adapter Swapping for Multi-Domain Serving

Netflix runs multiple content domains across 50+ markets (languages, content types, regional preferences). Rather than maintaining separate fine-tuned models per domain, Netflix serves a single quantized base model with domain-specific LoRA adapters loaded per-request. vLLM's LoRA adapter serving (--enable-lora flag) makes this efficient — the base model stays loaded in GPU memory while adapters are swapped in microseconds.

Section 10 — Text-to-SQL Thread: SQL Domain Adaptation

Piece 1 (Day 6) established the SQLOutput schema. Day 8 quantized the inference engine. Today we add the fine-tuning layer — adapting the base model to your specific SQL domain.

The Quality Gap Without Fine-Tuning

Model	Setup	Execution Accuracy on Domain SQL
LLaMA 3 70B base	Zero-shot, no schema context	~40-50%
LLaMA 3 70B base	Schema RAG (Week 3 Piece 2)	~60-70%
LLaMA 3 70B LoRA r=32	Schema RAG + 50K SQL examples	~88-92%
GPT-4o	Schema RAG, no fine-tuning	~85-90%

SQL Fine-Tuning Dataset Format

```
# SFT dataset: (schema + question) -> SQL pairs
# ChatML format for LLaMA-3-Instruct:

def format_sql_example(schema, question, sql):
    return (
        "<|im_start|>system\n"
        "You are a SQL expert. Generate valid SQL from the schema and question.\n"
        "Return only the SQL query, no explanation.<|im_end|>\n"
        "<|im_start|>user\n"
        f"Schema:\n{schema}\n\nQuestion: {question}<|im_end|>\n"
        "<|im_start|>assistant\n"
        f"{sql}<|im_end|>"
    )

# Example entry:
example = format_sql_example(
    schema="orders(id,user_id,total,created_at) users(id,email,country)",
    question="Total revenue by country in the last 30 days",
    sql=("SELECT u.country, SUM(o.total) AS revenue "
         "FROM orders o JOIN users u ON o.user_id = u.id "
         "WHERE o.created_at >= NOW() - INTERVAL 30 DAY "
         "GROUP BY u.country ORDER BY revenue DESC")
)
```

QLoRA Config for SQL

```
# SQL is knowledge-intensive -> use attention + FFN, higher rank
lora_config = LoraConfig(
    r=32,               # higher rank: SQL needs richer pattern storage
    lora_alpha=64,      # alpha = 2r
    target_modules=[    # attention + FFN both matter for SQL
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

# Training data recommendation for production SQL:
# Minimum: 5,000 validated (question, SQL) pairs
# Target:  50,000+ pairs covering diverse query patterns
# Include: simple selects, joins, aggregations, subqueries,
#          window functions, CTEs, dialect-specific syntax
# Validate: run generated SQL against test DB, check execution accuracy
```

WEEK 3 connects the remaining piece: Schema RAG retrieves the relevant tables and columns from a large warehouse schema (potentially 1,000+ tables) so the fine-tuned model sees only what it needs. LoRA adapter + Schema RAG together close the gap to GPT-4o quality at a fraction of the API cost.

Section 11 — Architect's Lens

```
DECISION FRAMEWORK: When to use LoRA?

1. Can you solve this with better prompting or RAG?
   YES -> Do that first. LoRA is not needed.
   NO  -> The model lacks a learnable skill. Continue.

2. How much labeled data do you have?
   < 100 examples    -> Too few for LoRA; use few-shot prompting
   100 - 1,000       -> LoRA r=8, small model, 1-3 epochs, careful eval
   1,000 - 10,000    -> LoRA r=16, standard recipe
   10,000+           -> LoRA r=32-64, or consider full fine-tuning

3. What is your compute constraint?
   Consumer GPU (RTX 3090 24 GB)  -> QLoRA r=8-16 on 7B model
   One A100 40 GB                 -> QLoRA r=16 on 13B, or 70B at r=8
   One A100 80 GB                 -> QLoRA r=16-32 on 70B model
   4+ A100s                       -> LoRA r=64 on 70B, or Full FT on 7B

4. How will you serve the adapter?
   Single task      -> Merge + AWQ quantize (zero overhead)
   Multiple tasks   -> Keep adapters separate, vLLM --enable-lora
   Edge/offline     -> Merge + GGUF Q4_K_M

5. Quality vs cost priority?
   Cost-first       -> QLoRA r=16 smallest viable model + merge + AWQ
   Quality-first    -> Full fine-tuning large model, distill to smaller
```

The Quality Data Trap

Fine-tuning on 500 noisy examples is worse than not fine-tuning at all. LoRA amplifies the signal in your data — if the signal is noise, you get a noisier model. Always validate training data quality before training. A 1,000-example validated dataset beats a 10,000-example scraped dataset every time.

LoRA Rank and Task Complexity

Task Type	Recommended r	Why
Binary classification	4–8	Minimal behavior change needed
Named entity extraction	8–16	Format learning, low complexity
SQL generation	16–32	Pattern-rich, knowledge-intensive
Multilingual adaptation	32–64	Many independent directions needed
Personality/style shift	32–64	Broad behavior change across layers

Safety alignment	Full FT + DPO	LoRA insufficient for deep alignment

Section 12 — Hands-On Exercises

Exercise 1: Count LoRA Parameters

See how rank directly controls trainable parameters:

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM
import torch

model_name = "meta-llama/Llama-3.2-1B"  # small — loads on CPU
model = AutoModelForCausalLM.from_pretrained(model_name,
            torch_dtype=torch.float16, device_map="cpu")

for r in [4, 8, 16, 32]:
    config = LoraConfig(
        r=r, lora_alpha=r*2,
        target_modules=["q_proj", "v_proj"],
        bias="none", task_type=TaskType.CAUSAL_LM,
    )
    lora_model = get_peft_model(model.base_model.model, config)
    print(f"\n--- rank={r} ---")
    lora_model.print_trainable_parameters()
    # Expected output pattern:
    # trainable params: X || all params: Y || trainable%: Z%
```

Exercise 2: Build a SQL Fine-Tuning Dataset

Generate training pairs using Claude then format for SFT:

```
import anthropic, json

client = anthropic.Anthropic()

SCHEMA = """
orders (id, user_id, product_id, quantity, total_price, created_at)
users (id, email, country, signup_date)
products (id, name, category, price, stock_quantity)
"""

def generate_sql_pairs(schema, n=10):
    resp = client.messages.create(
        model="claude-haiku-4-5-20251001",
        max_tokens=2000,
        messages=[{"role": "user", "content":
            f"Generate {n} diverse natural language questions + correct SQL "
            f"for this schema:\n{schema}\n"
            f'Return JSON array: [{{"question":"...","sql":"SELECT..."}}]'}]
    )
```

```
    return json.loads(resp.content[0].text)

def to_chatml(pair, schema):
    return {"text": (
        "<|im_start|>system\nGenerate SQL for the schema and question.<|im_end|>\n"
        f"<|im_start|>user\nSchema: {schema}\nQ: {pair['question']}<|im_end|>\n"
        f"<|im_start|>assistant\n{pair['sql']}<|im_end|>"
    )}

pairs = generate_sql_pairs(SCHEMA, n=20)
sft_data = [to_chatml(p, SCHEMA) for p in pairs]
print(f"Generated {len(sft_data)} training examples")
for ex in sft_data[:2]:
    print(ex["text"][:200])
```

Exercise 3: Scenario Reasoning

Scenario A: Fine-tune LLaMA 3 70B for SQL on a single A100 80GB.

Answer: QLoRA NF4 r=16 on attention + FFN. 70B NF4 = 35 GB. Adapter + gradients = ~1 GB. Total ~36 GB with room for activations + KV cache.

Scenario B: Serve 3 different LoRA adapters (SQL, legal, medical) on one GPU.

Answer: Keep adapters separate. Use vLLM with --enable-lora. Base model loaded once in GPU memory; adapters hot-swapped per request.

Scenario C: Produce a single deployable model file from a QLoRA-trained adapter.

Answer: merge_and_unload() -> merged FP16 model. Then AWQ 4-bit quantize. Result: one .safetensors model, ~38 GB for 70B.

Scenario D: Your 500-example LoRA model performs worse than the base model.

Answer: Classic noisy-data failure. Audit your training examples for quality. 500 examples requires every single one to be correct and representative.

Summary — Day 9 Key Concepts

Concept	One-Line Definition
Low-rank hypothesis	Fine-tuning updates ΔW live in a tiny subspace of full weight space
LoRA	Decompose ΔW = B×A; train only B and A (0.5% of params)
Rank r	Dimension of low-rank subspace; r=16 is the recommended default
Alpha (α)	Scaling factor; α=2r so update scale = 2.0; higher = stronger update
QLoRA	NF4 base model (frozen) + LoRA adapters (trained): 70B on 1× A100
PEFT library	HuggingFace library: get_peft_model(), print_trainable_parameters()
merge_and_unload()	Bake W'=W+BA into base weights; zero inference overhead after merge
Adapter swapping	Keep adapters separate; swap per request for multi-task serving
AdaLoRA	Google's extension: auto-allocate rank per layer by importance score
Full FT vs LoRA	LoRA: 90-95% quality at 0.5% cost; Full FT: max quality, max cost

TOMORROW — Day 10: Full Fine-Tuning

When is LoRA not enough? Day 10 covers full fine-tuning: when to commit to training all parameters, the complete DeepSpeed + FSDP stack, and the definitive decision framework — prompt engineering vs RAG vs LoRA vs full fine-tuning vs pre-training.

EAGLE EYE ADDENDUM — DAY 9: Catastrophic Forgetting in LoRA Fine-Tuning

Day 9 covered LoRA math, QLoRA, and the PEFT library. One important risk was missing: catastrophic forgetting — the degree to which fine-tuning overwrites the base model's general capabilities.

Catastrophic Forgetting and LoRA

Catastrophic forgetting occurs when a model trained on domain-specific data loses general capabilities it acquired during pre-training. LoRA significantly reduces this risk compared to full fine-tuning — but does not eliminate it entirely.

Why LoRA Has Lower Forgetting Risk Than Full FT

LoRA trains only 0.5–2% of parameters (the adapter matrices A and B). The base model weights W are frozen — they cannot change. Only the low-rank delta W' = W + (alpha/r) * BA is active. Because W never moves, the vast majority of pre-trained knowledge is preserved in the frozen weights.

```
Forgetting risk by method (relative scale):

Method                    Trainable Params   Forgetting Risk
Full fine-tuning          100%               High (all weights can shift)
LoRA r=64                 ~2%                Moderate (larger adapters)
LoRA r=16 (standard)      ~0.5%             Low (adapters are small)
LoRA r=4                  ~0.1%             Very low
Prefix Tuning             ~0.1%             Very low (weights frozen)
IA3 / Prompt Tuning       ~0.01%            Negligible

Key insight: LoRA r=16 gives the best forgetting/quality trade-off.
LoRA r=64 approaches full FT in both quality AND forgetting risk.
```

When LoRA CAN Cause Forgetting

Despite frozen base weights, LoRA fine-tuning can still degrade general capabilities in three situations:

- High rank (r=64+) + high alpha (128+): adapter delta becomes large enough to dominate W, effectively shifting behaviour far from pre-trained distribution
- Very high learning rate (>5e-4): adapters diverge rapidly in early epochs, overfitting to domain data and suppressing general capability
- Tiny dataset (<500 examples): model memorises training examples rather than generalising — general capability replaced by rote patterns

Prevention Strategies for LoRA

```
# Strategy 1: Conservative hyperparameters (most important)
lora_config = LoraConfig(
    r=16,              # do not exceed 32 unless quality ceiling proven
    lora_alpha=32,     # keep alpha = 2r convention
    lora_dropout=0.05, # small dropout regularises adapters
```

```
    bias="none",
)
training_args = TrainingArguments(
    learning_rate=2e-4,    # LoRA standard; do NOT use full-FT rates (2e-5)
    num_train_epochs=3,    # evaluate after each epoch; stop early if general
                           # capability degrades on held-out general bench
)

# Strategy 2: Data mixing (same as full FT, but smaller proportion needed)
train_data = concatenate_datasets([
    domain_dataset.select(range(9_000)),         # 95% domain
    general_alpaca_dataset.select(range(500)),   # 5% general anchor
])
# For LoRA, 5% general data is usually sufficient (vs 10-20% for full FT)

# Strategy 3: Evaluate on BOTH domain AND general benchmarks
# After fine-tuning, check:
# - Domain task: SQL execution accuracy (target metric)
# - General task: MMLU subset / MT-Bench (should not drop >5%)
```

LoRA Forgetting vs Full FT Forgetting

Scenario	LoRA r=16	Full Fine-Tuning
SQL acc. gained	+25-30%	+28-33%
General Q&A; degradation	1-3%	5-15% (without data mixing)
Recovery method	Reduce rank or LR	Data mixing + weight decay (Day 10)
Worst case	Adapter overfits; merge + retry	Full retraining required

ARCHITECT RULE: After any LoRA fine-tuning run, evaluate the merged model on a small general benchmark (10-20 diverse questions from MMLU or MT-Bench) alongside your domain task. A general capability drop > 5% signals forgetting and usually means the rank or learning rate is too high. LoRA r=16 with lora_alpha=32 and lr=2e-4 rarely causes meaningful forgetting.